

40. To put this number in perspective, OpenSolaris provides more than 200 system calls for applications, to access kernel services.

SMF @ work

If the following six SMF services are online, that means we are ready to install new guests on this Xen system. You can run `'svcs -p'` against each of those services to see what processes are associated with those services (for example, `#svcs -p xvm/store`). As these long-running daemons are wrapped as SMF services, benefits like an automatic restart in case of unexpected termination, follow.

```
surya@opensolaris:~$ svcs -a | grep xvm
disabled Jun_26 svc:/system/xvm/ipagent:default
online Jun_26 svc:/system/xvm/vm-config:default
online Jun_26 svc:/system/xvm/store:default
online Jun_26 svc:/system/xvm/xend:default
online Jun_26 svc:/system/xvm/console:default
online Jun_26 svc:/system/xvm/virtld:default
online Jun_26 svc:/system/xvm/domains:default
surya@opensolaris:~$
```

Note that `'ipagent'` doesn't run in dom0; it runs only in PV domUs.

The power of ZFS

We could create a ZFS volume and pass it as an install disk to a guest and reap the benefits associated with this modern filesystem, such as cloning (we can deploy many more VMs after modifying per instance data), snapshots, end-to-end data integrity, unlimited space, etc.

We use `virt-install` (1) to install new guests. Please follow the instructions at http://opensolaris.org/os/community/xen/docs/opensolaris_domu to install OpenSolaris 2009.06 as a PV guest. After a successful install and reboot of the new domU, from dom0 we can list out the currently installed domains, using the following:

```
surya@opensolaris:~$ pfexec virsh list
Id Name State
-----
0 Domain-0 running
1 osol blocked
surya@opensolaris:~$
```

As you can see, I have named my OpenSolaris 2009.06 guest as 'osol'. Now you can take the domU console using `'virsh console'`. Log in to the domU using the account we configured while installing. Log in and confirm that it is not a privileged domain.

Thus we now have a domU running besides dom0—i.e., two operating systems running simultaneously on a

single physical machine—Xen at work!

Dynamically, we can add more disk capacity to the osol domU by passing additional files or block devices from dom0.

First, confirm from inside domU, how many block devices are available using the `format (1)` command. We can use either `'virsh (1M)'` or `'xm (1M)'` to attach new drives from dom0. `xm (1M)` stands for Xen Manager and is the original tool that came along with Xen. It provides command and control of the hypervisor. This is being deprecated in favour of `virsh`, albeit in places where `virsh` hasn't yet provided the same functionality.

```
surya@opensolaris:~$ mkfile 100M /var/tmp/disk2
surya@opensolaris:~$ pfexec chown xvm /var/tmp/disk2
surya@opensolaris:~$ pfexec xm block-attach osol tap:vdisk:/var/tmp/
disk2 3 w
surya@opensolaris:~$
surya@opensolaris:~$ pfexec zfs create -V 5G rpool/xvm-drive3
surya@opensolaris:~$ pfexec virsh attach-disk osol /dev/zvol/dsk/rpool/
xvm-drive3 xvdd --driver phy
surya@opensolaris:~$
```

The above commands add a regular file and ZFS volume to the domU as additional disks.

From dom0, we could use `'virsh dumpxml osol'` and spot the new disks that we just added. From domU, we could run the `format (1M)` command and spot the presence of additional drives. We could run `pfexec virsh detach-disk xvdd` and see one of the drives vanish from domU.

As can be seen, the changes are reflected in the domU immediately.

Similarly, additional vNICs can also be passed onto the running guest. Likewise, vCPUs can also be taken off the running domain. But to pass more vCPUs than what is already configured, domU has to be shut down and its config file updated—i.e., run `'virsh dumpxml osol > /tmp/osol.xml'` and try changing vCPUs in the `/tmp/osol.xml` file, then undefine and redefine the guest using the new `/tmp/osol.xml` file.

Secure by default

Look for a process called `'libvirt'` in dom0 and `truss (1)` it:

```
surya@opensolaris:~$ pfexec truss -i -o /tmp/foo -p 'pgrep libvirt'
```

Now, from another dom0 terminal, execute `pfexec virsh list`, and the above truss starts tracing all the system calls being made by `libvirt`. So what is happening? All the `libvirt` tools communicate with `xend` via `libvirt`—the `libvirt` process is made to run with `least privileges (5)`, whereas `virsh (1M)` and family are not.

We could run `ps -f -u xvm` in dom0 and confirm that none of the services associated with Xen run